Basics of Computer Organization

DSC 204A: Scalable Data Systems

Haojian Jin

Where we are in the class?

Foundations of Data Systems

- Data representation & encoding
- Basics of Computer Organization
- Operating Systems
- **Storage & Memory Hierarchy**
- Cache

Scaling & Distributed Systems

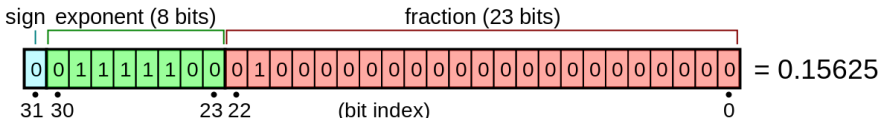
- Encoding & evolution
- Distributed Processing
- Programming model

Machine Learning Systems

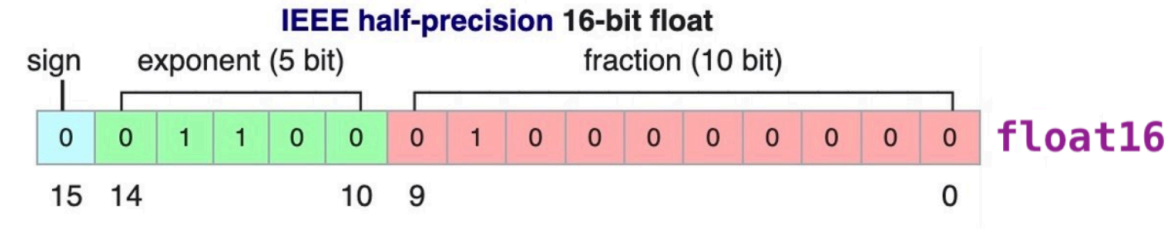
- ML Systems
- Batch & Stream Processing

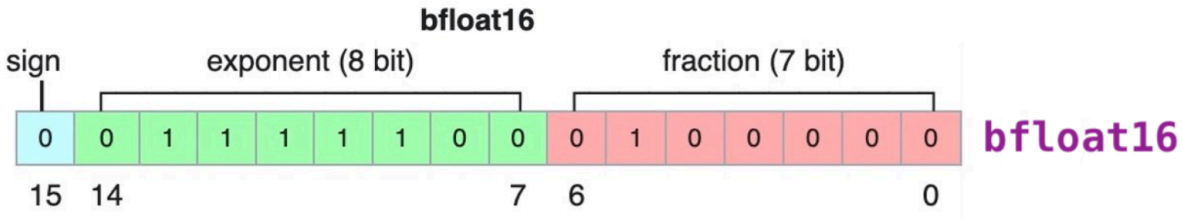
Review: What does exponent and fraction control?

- **Exponent** controls: range, offset
- **Fraction** controls: actual value, precision



Review: BF16 vs. FP16





- Less exponent → smaller range → **easier to overflow**
- More exponent → larger range → **harder to overflow**
- More fraction → **more precise**
- Less fraction → **less precise**

Review: Latest FP8

- Exponent width → **Range**; Fraction width → **Precision**

	Exponent (bits)	Fraction (bits)	Total (bits)
IEEE 754 Single Precision 32-bit Float (IEEE FP32) 	8	23	32
IEEE 754 Half Precision 16-bit Float (IEEE FP16) 	5	10	16
Nvidia FP8 (E4M3)  * FP8 E4M3 does not have INF, and S.1111.111$_2$ is used for NaN. * Largest FP8 E4M3 normal value is S.1111.110$_2$ =448.	4	3	8
Nvidia FP8 (E5M2) for gradient in the backward  * FP8 E5M2 have INF (S.11111.00$_2$) and NaN (S.11111.XX$_2$). * Largest FP8 E5M2 normal value is S.11110.11$_2$ =57344.	5	2	8

Review: FP4

NVIDIA 2025-9-30

Pretraining Large Language Models with NVFP4

NVIDIA

Abstract. Large Language Models (LLMs) today are powerful problem solvers across many domains, and they continue to get stronger as they scale in model size, training set size, and training set quality, as shown by extensive research and experimentation across the industry. Training a frontier model today requires on the order of tens to hundreds of yottaflops, which is a massive investment of time, compute, and energy. Improving pretraining efficiency is therefore essential to enable the next generation of even more capable LLMs. While 8-bit floating point (FP8) training is now widely adopted, transitioning to even narrower precision, such as 4-bit floating point (FP4), could unlock additional improvements in computational speed and resource utilization. However, quantization at this level poses challenges to training stability, convergence, and implementation, notably for large-scale models trained on long token horizons.

In this study, we introduce a novel approach for stable and accurate training of large language models (LLMs) using the NVFP4 format. Our method integrates Random Hadamard transforms (RHT) to bound block-level outliers, employs a two-dimensional quantization scheme for consistent representations across both the forward and backward passes, utilizes stochastic rounding for unbiased gradient estimation, and incorporates selective high-precision layers. We validate our approach by training a 12-billion-parameter model on 10 trillion tokens – the longest publicly documented training run in 4-bit precision to date. Our results show that the model trained with our NVFP4-based pretraining technique achieves training loss and downstream task accuracies comparable to an FP8 baseline. For instance, the model attains an MMLU-pro accuracy of 62.58%, nearly matching the 62.62% accuracy achieved through FP8 pretraining. These findings highlight that NVFP4, when combined with our training approach, represents a major step forward in narrow-precision LLM training algorithms.

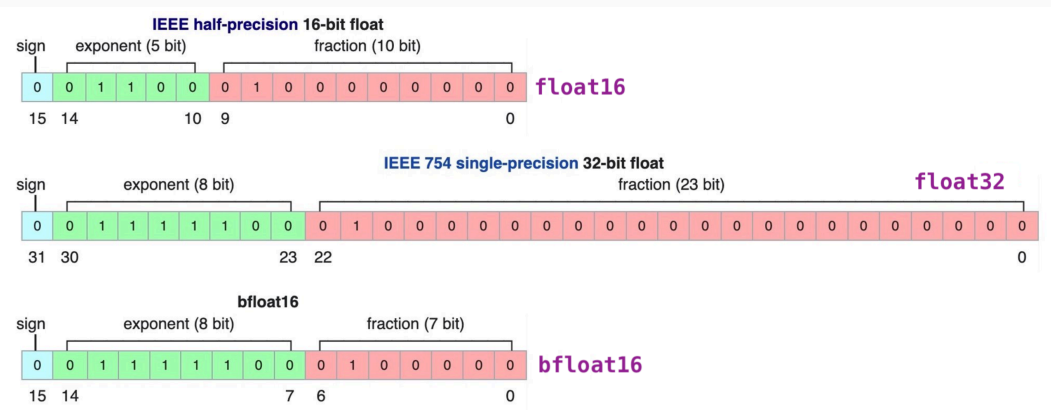
Code: Transformer Engine support for NVFP4 training.

1. Introduction

The rapid expansion of large language models (LLMs) has increased the demand for more efficient numerical formats to lower computational cost, memory demand, and energy consumption during training. 8-bit floating point (FP8 and MXFP8) has emerged as a popular data type for accelerated training of LLMs (Micikevicius et al., 2022; DeepSeek-AI et al., 2024; Mishra et al., 2025). Recent advances in narrow-precision hardware (NVIDIA Blackwell, 2024) have positioned 4-bit floating point (FP4) as the next logical step (Tseng et al., 2025b; Chmiel et al., 2025; Wang et al., 2025; Chen et al., 2025; Castro

Introducing NVFP4 for Efficient and Accurate Low-Precision Inference

Review: Why BF16 is better in ML/AI?

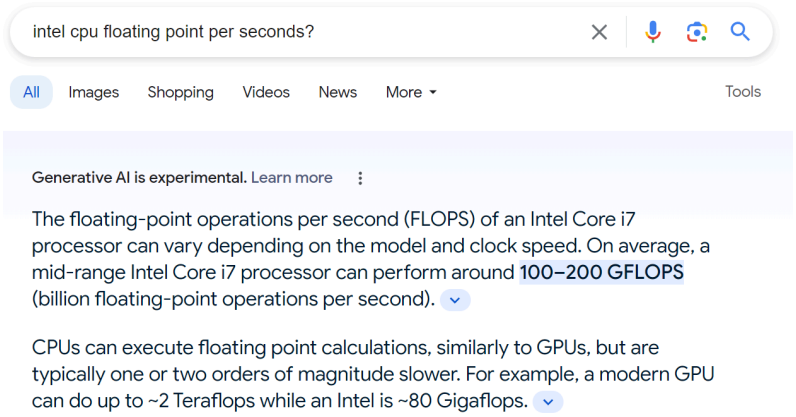


- **Precision is enough** – ML/AI is error-tolerant (why?)
- **Deep learning is easy to overflow**
- **Closer range to FP32**

Review: How Fast is a Processor (CPU and GPU)?

- **Instructions/second**: number of instructions a processor can do
- Data science: We care more about computation on floating point numbers
- **FLOPS**: number of floating point operations a processor can do

Form Factor	H100 SXM
FP64	34 teraFLOPS
FP64 Tensor Core	67 teraFLOPS
FP32	67 teraFLOPS
TF32 Tensor Core	989 teraFLOPS[2]
BFLOAT16 Tensor Core	1,979 teraFLOPS[2]
FP16 Tensor Core	1,979 teraFLOPS[2]
FP8 Tensor Core	3,958 teraFLOPS[2]



copyij vs. copyji: Copy a 2048×2048 integer array

```
void copyij(long int src[2048][2048], long int dst[2048][2048])
{
  long int i,j;
  for (i = 0; i < 2048; i++)
    for (j = 0; j < 2048; j++)
      dst[i][j] = src[i][j];
}

void copyji(long int src[2048][2048], long int dst[2048][2048])
{
  long int i,j;
  for (j = 0; j < 2048; j++)
    for (i = 0; i < 2048; i++)
      dst[i][j] = src[i][j];
}
```

`copyij` : **4.3 ms** `copyji` : **81.8 ms**

The Real Problem? The Physics of Computing.

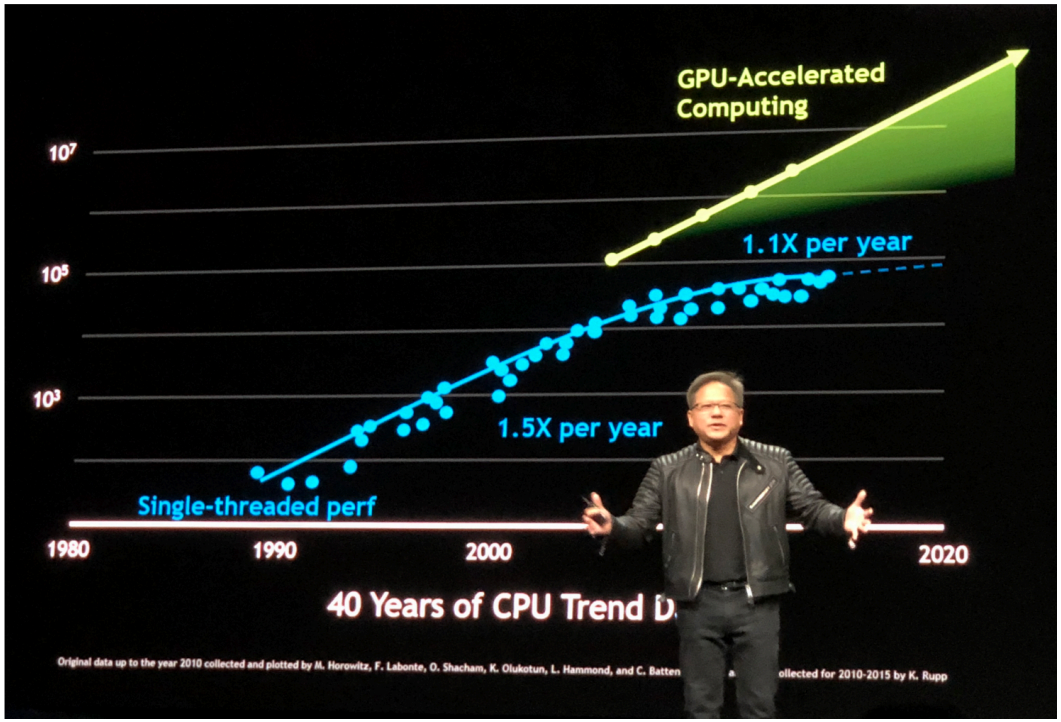
- CPU: **100 GFLOPs/s**
- Assume we use 0.5s to perform 50 GFLOPs
- We need to read 50×2 = **100 GB** in the remaining 0.5s to keep the CPU busy
- We need the CPU to read at a speed of 100 GB / 0.5s = **200 GB/s**
- But disk speed is only **80–160 MB/s**

Memory/Storage Hierarchy

	Speed	Capacity	Cost
Registers/Cache	~100 GB/s	~MBs	~$2/MB
DRAM	~10 GB/s	~10 GBs	~$5/GB
Non-Volatile RAM	~GB/s	–	–
SSD	~200 MB/s	~TBs	~$200/TB
HDD	~50 MB/s	~10 TBs	~$30/TB
Cloud/Tape	–	~PBs	~$10/TB

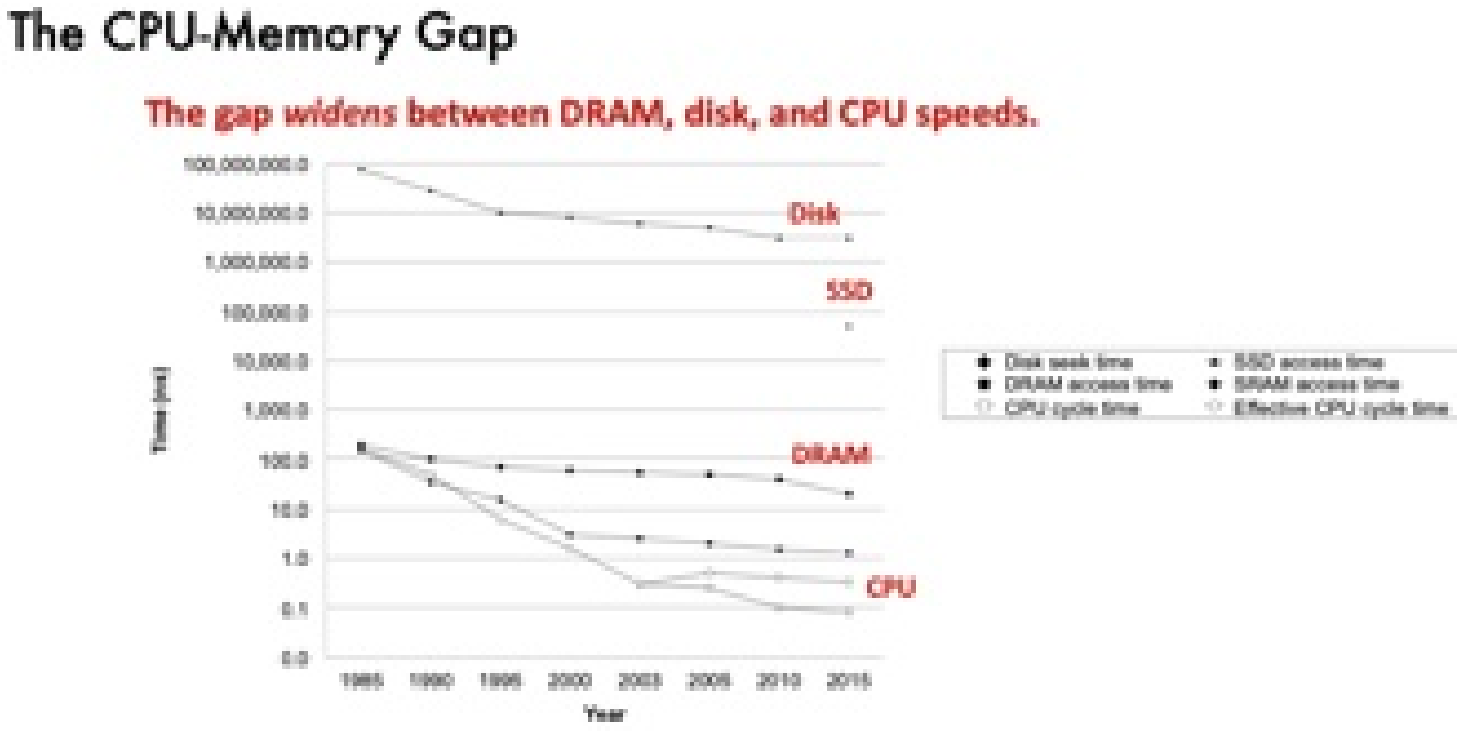
Moore's Law

> "The number of transistors on a microchip doubles approximately every two years." – Gordon Moore, 1965



The CPU-Memory Gap

The gap widens between DRAM, disk, and CPU speeds.



Locality

> The key to bridging this CPU-Memory gap is an important property of computer programs known as **locality**.

Locality

- **Principle of Locality**: Many programs tend to use data and instructions with addresses near or equal to those they have used recently
- **Temporal locality**:
  - Recently referenced items are likely to be referenced again in the near future
- **Spatial locality**:
  - Items with nearby addresses tend to be referenced close together in time
- Both hardware reason and software reason.

Locality Example

```
num_list = [1, 2, 3, 4, 5, 7]
sum = 0
for x in num_list:
    sum += x
return sum
```

- Data references:
  - Reference array elements in succession (stride-1) → **spatial**
  - Reference variable `sum` each iteration → **temporal**
- Instruction references:
  - Reference instructions in sequence → **spatial**
  - Cycle through loop repeatedly → **temporal**

Qualitative Estimates of Locality

Q: Does this function have good locality w.r.t. array `a`?

```
int sum_array_rows(int a[M][N]) {
    int i, j, sum = 0;
    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            sum += a[i][j];
    return sum;
}
```

Answer: Yes — accesses row-major order (stride-1 pattern)

Locality Example

Q: Does this function have good locality w.r.t. array `a`?

```
int sum_array_cols(int a[M][N]) {
    int i, j, sum = 0;
    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            sum += a[i][j];
    return sum;
}
```

Answer: No — accesses column-major order (stride-N pattern), unless M is very small

Example Exam Question

Q: Can you permute the loops so that the function scans the 3D array `a` with a stride-1 reference pattern (and thus has good spatial locality)?

```
int sum_array_3d(int a[M][N][N]) {
    int i, j, k, sum = 0;
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            for (k = 0; k < M; k++)
                sum += a[k][i][j];
    return sum;
}
```

Example Exam Question: Answer

Answer: make j the inner loop

```
int sum_array_3d(int a[M][N][N]) {
    int i, j, k, sum = 0;
    for (k = 0; k < M; k++)
        for (i = 0; i < N; i++)
            for (j = 0; j < N; j++)
                sum += a[k][i][j];
    return sum;
}
```

- `a[M][N][N]` is stored in row-major order: last index `j` varies fastest in memory
- Making `j` the innermost loop → stride-1 access pattern → good spatial locality

Why?

Reason 1: Software Cache

Cache in action

Cache

Smaller, faster, more expensive memory caches a subset of the blocks

Memory

Larger, slower, cheaper memory viewed as partitioned into "blocks"

Data is copied in **block-sized transfer units**

General Cache Concepts: Hit

- Data in block **14** is needed
- **Request: 14**
- Block 14 **is** in cache: Hit!

Cache:

8	9	14	3

General Cache Concepts: Miss

- Data in block **12** is needed
- **Request: 12**
- Block 12 is **not** in cache: Miss!
- Block 12 is fetched from memory
- Block 12 is stored in cache
  - **Placement policy**: where block goes
  - **Replacement policy**: which block gets evicted

Cache (after):

8	12	14	3

Cache in action

Processor → *~100 GB/s* → **Cache** → *~10 GB/s* → **Memory**

- If **always cache hit**, bandwidth? → **~100 GB/s**
- If **always cache miss**, bandwidth? → **~10 GB/s**

Open Question in Cache: ChatGPT

Processor → *~100 GB/s* → **Cache** → *~10 GB/s* → **Memory**

- Parameters: **350 GB**
- ChatGPT: every time it outputs a token, it needs to see **350 GB** of parameters
- **How to optimize this?**

3 Types of Cache Misses

- **Cold (compulsory) miss**
  - Cache starts empty — this is the first reference to the block
- **Capacity miss**
  - The set of active cache blocks (working set) is larger than the cache
- **Conflict miss**
  - Most caches limit blocks at level k+1 to a small subset of block positions at level k
  - E.g., block i at level k+1 must be placed in block (i mod 4) at level k
  - Conflict misses occur when the cache is large enough, but multiple data objects all map to the same block
  - E.g., referencing blocks 0, 8, 0, 8, 0, 8, ... would miss every time

Why?

Reason 2: Hardware Design

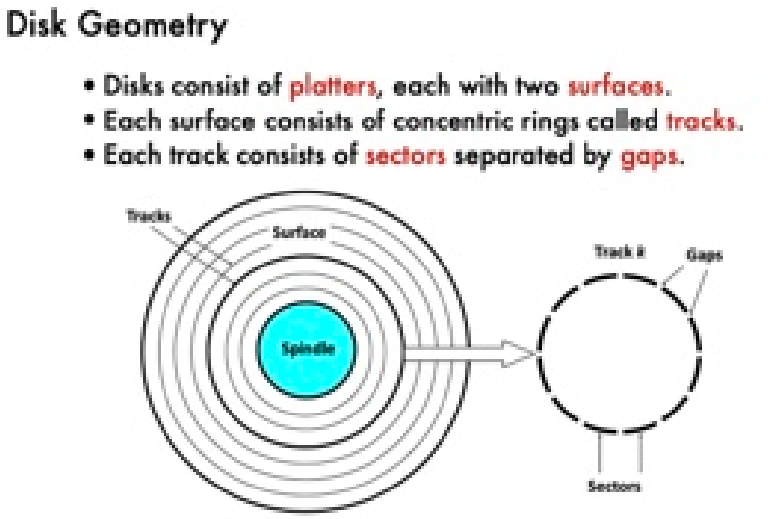
What's Inside A Disk Drive?

Spindle, Platters, Actuator, Read/write heads, Electronics (including a processor and memory!)

Image courtesy of Seagate Technology

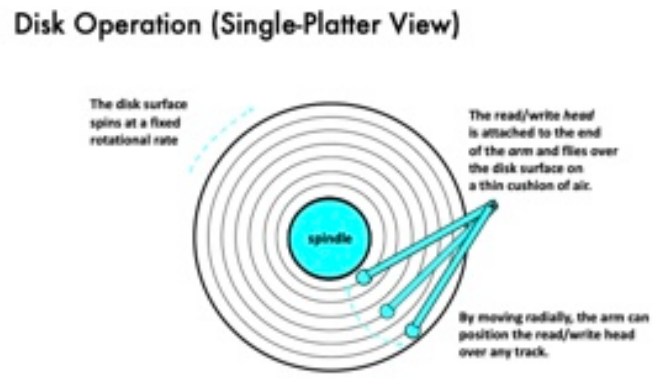
Disk Geometry

- Disks consist of **platters**, each with two **surfaces**
- Each surface consists of concentric rings called **tracks**
- Each track consists of **sectors** separated by **gaps**



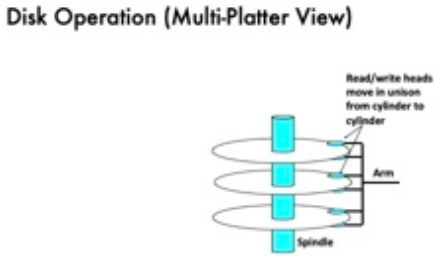
Disk Operation (Single-Platter View)

- The disk surface spins at a fixed **rotational rate**
- The read/write head is attached to the end of the arm and flies over the disk surface on a thin cushion of air
- By moving radially, the arm can position the read/write head over any track



Disk Operation (Multi-Platter View)

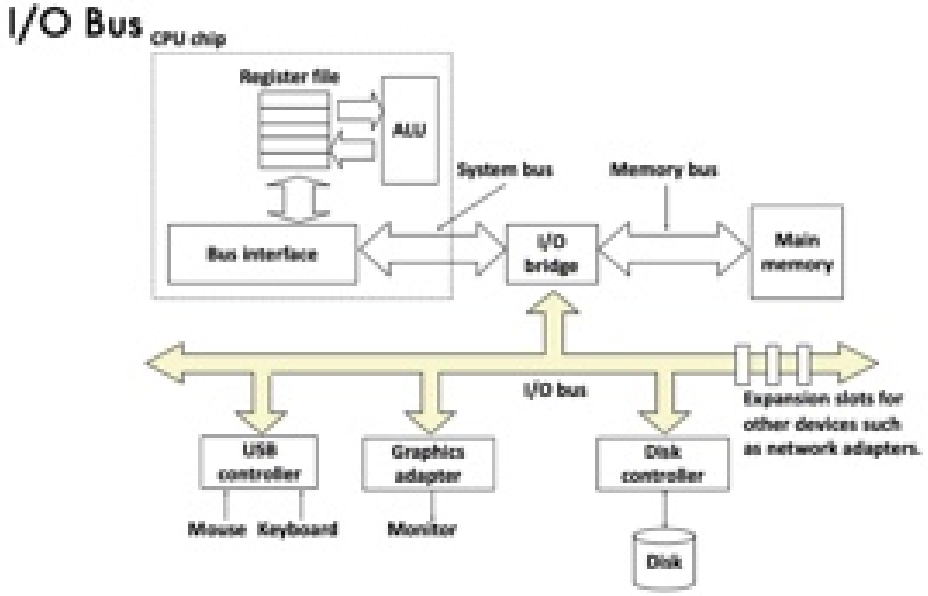
- Read/write heads move in unison from cylinder to cylinder



Disk Capacity

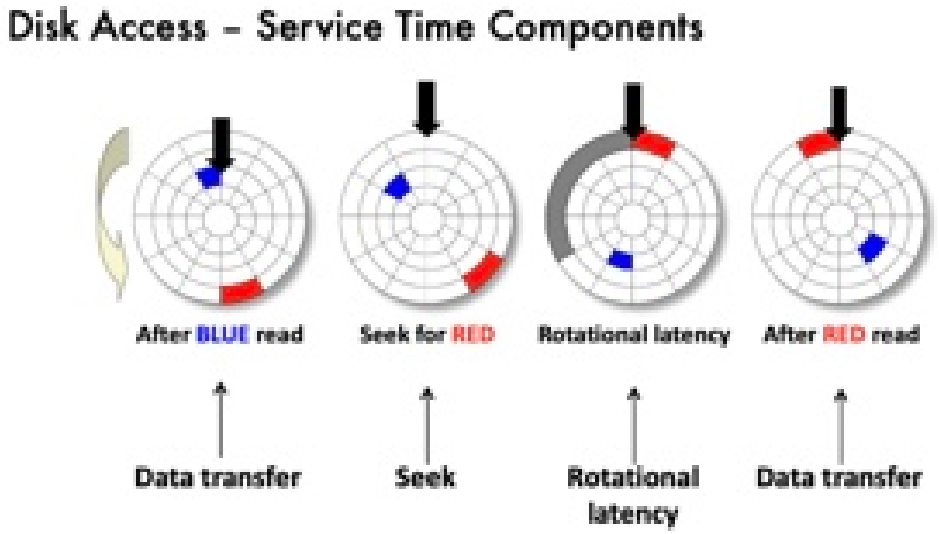
- **Capacity**: maximum number of bits that can be stored
  - Vendors express capacity in GB (10^9 bytes) or TB (10^{12} bytes)
- Capacity is determined by:
  - **Recording density** (bits/in): bits squeezed into a 1 inch segment of a track
  - **Track density** (tracks/in): tracks squeezed into a 1 inch radial segment
  - **Areal density** (bits/in^2): product of recording and track density

I/O Bus



Expansion slots for other devices such as network adapters.

Disk Access — Service Time Components



Data transfer → Seek → Rotational latency → Data transfer

Disk Access Time

Average time to access some target sector:

T_access = T_avg seek + T_avg rotation + T_avg transfer

- **Seek time** (T_avg seek)
  - Time to position heads over cylinder containing target sector
  - Typical: **3–9 ms**
- **Rotational latency** (T_avg rotation)
  - Time waiting for first bit of target sector to pass under r/w head
  - T_avg rotation = 1/2 × 1/RPM × 60 sec/min
- **Transfer time** (T_avg transfer)
  - Time to read the bits in the target sector
  - T_avg transfer = 1/RPM × 1/(avg # sectors/track) × 60 sec/min

Disk Access Time Example

Given: Rotational rate = 7,200 RPM, Avg seek time = 9 ms, Avg # sectors/track = 400

Derived:

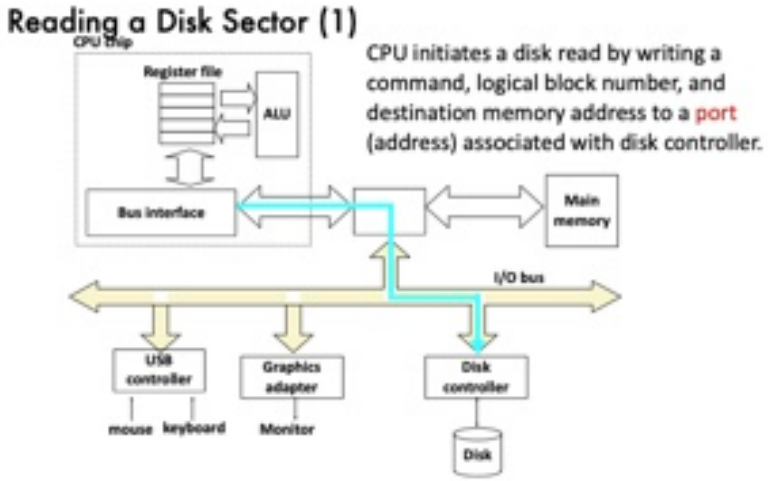
- T_avg rotation = 1/2 × (60s / 7200 RPM) × 1000 ms/s = **4 ms**
- T_avg transfer = 60/7200 × 1/400 × 1000 ms/s = **0.02 ms**
- **T_access = 9 ms + 4 ms + 0.02 ms ≈ 13 ms**

Important points:

- Access time dominated by **seek time** and **rotational latency**
- First bit in a sector is the most expensive, the rest are free
- SRAM access time ≈ 4 ns, DRAM ≈ 60 ns
- Disk is about **40,000× slower than SRAM**, **2,500× slower than DRAM**

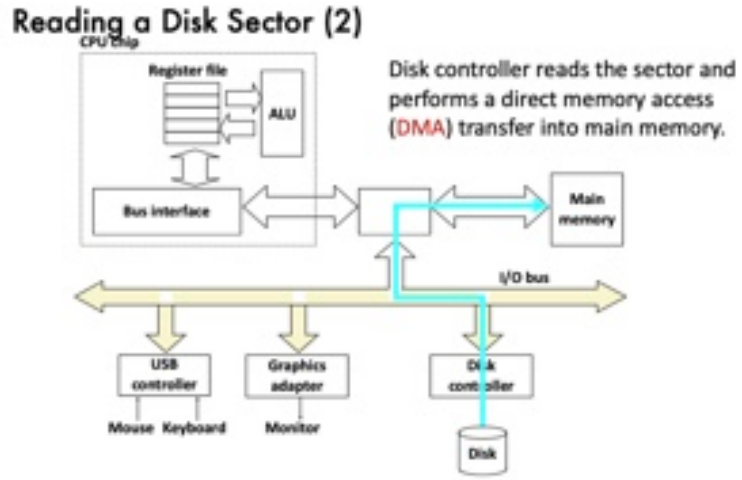
Reading a Disk Sector (1)

CPU initiates a disk read by writing a command, logical block number, and destination memory address to a port (address) associated with disk controller.



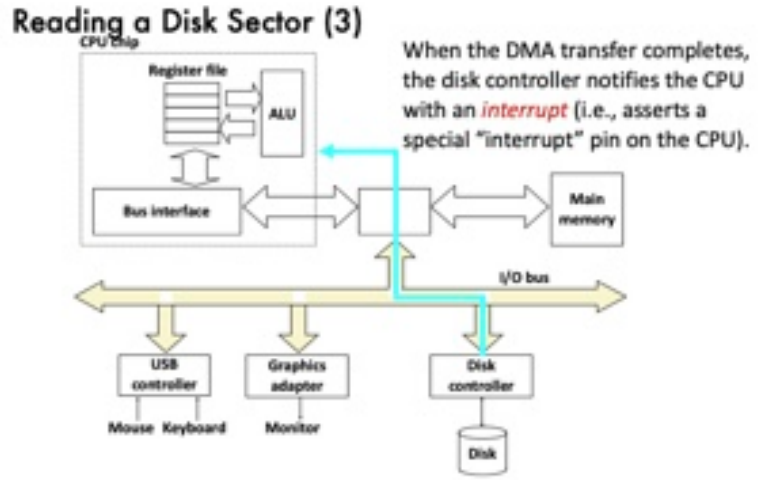
Reading a Disk Sector (2)

Disk controller reads the sector and performs a **direct memory access (DMA)** transfer into main memory.



Reading a Disk Sector (3)

When the DMA transfer completes, the disk controller notifies the CPU with an **interrupt** (i.e., asserts a special "interrupt" pin on the CPU).



Memory Hierarchy

Memory Hierarchies

Some fundamental and enduring properties of hardware and software:

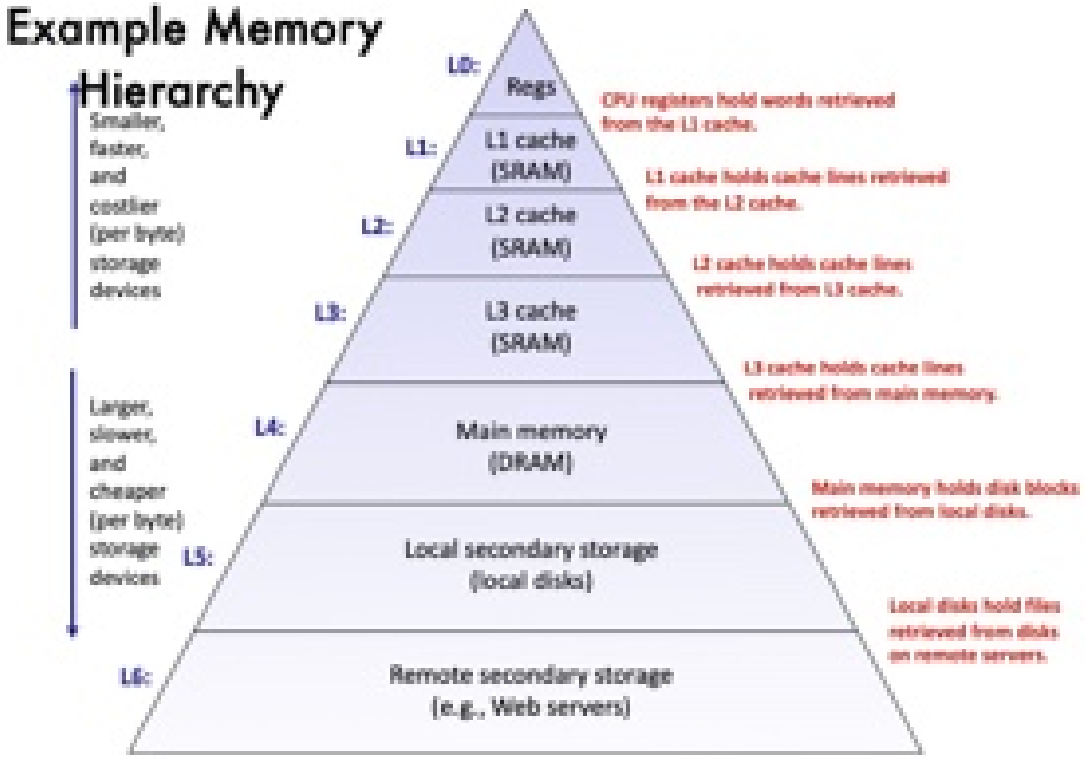
- Fast storage technologies **cost more per byte**, have less capacity, and require more power (heat!)
- The **gap between CPU and main memory speed** is widening
- Well-written programs tend to exhibit **good locality**

These properties suggest an approach for organizing memory and storage systems known as a **memory hierarchy**.

Example Memory Hierarchy

Level	Storage	What it caches	Speed
L0	**Registers**	Words from L1 cache	Fastest
L1	**L1 Cache (SRAM)**	Cache lines from L2	
L2	**L2 Cache (SRAM)**	Cache lines from L3	
L3	**L3 Cache (SRAM)**	Cache lines from main memory	
L4	**Main Memory (DRAM)**	Disk blocks from local disks	
L5	**Local Disks**	Files from remote servers	
L6	**Remote Storage**	—	Slowest

Example Memory Hierarchy



Caches

- **Cache**: A smaller, faster storage device that acts as a staging area for a subset of the data in a larger, slower device
- Fundamental idea of a memory hierarchy:
  - For each k, the faster, smaller device at level k serves as a **cache** for the larger, slower device at level k+1
- **Why do memory hierarchies work?**
  - Because of **locality**: programs tend to access the data at level k more often than level k+1
  - Thus, storage at level k+1 can be slower, and thus larger and cheaper per bit
- **Big Idea (Ideal):** The memory hierarchy creates a large pool of storage that costs as much as the cheap storage near the bottom, but serves data at the rate

Examples of Caching in the Memory Hierarchy

Cache Type	What is Cached?	Where Cached?	Latency (cycles)	Managed By
Registers	4-8 byte words	CPU core	0	Compiler
L1 Cache	64-byte blocks	On-chip L1	4	Hardware
L2 Cache	64-byte blocks	On-chip L2	10	Hardware
Virtual Memory	4-KB pages	Main memory	100	Hardware + OS
Buffer Cache	Parts of files	Main memory	100	OS
Disk Cache	Disk sectors	Disk controller	100,000	Disk firmware
Network Buffer	Parts of files	Local disk	10,000,000	NFS client
Browser Cache	Web pages	Local disk	10,000,000	Web browser
Web Cache	Web pages	Remote server disks	1,000,000,000	Web proxy server

Memory Hierarchy in Practice: Pandas vs. Dask

- **Pandas** DataFrame needs data to fit entirely in DRAM
- **Dask** DataFrame automatically manages Disk vs DRAM for you
  - Full data sits on Disk, brought to DRAM upon `compute()`
  - Dask stages out computations using Pandas

Tradeoff: Dask may throw **memory configuration issues** :)

Processor Performance

Q: *How fast can a processor process a program?*

- Modern CPUs can run millions of instructions per second!
  - ISA tells #**clock cycles** per instruction
  - CPU's **clock rate** helps map that to runtime (ns)
- Alas, most programs do not keep CPU always busy:
  - Memory access commands **stall** the processor; ALU and CU are *idle* during DRAM-register transfer
  - Worse, data may not be in DRAM — wait for (disk) I/O!
  - So, actual *runtime* of program may be OOM higher than what clock rate calculation suggests

> **Key Principle:** Optimizing access to DRAM and use of processor caches is critical for processor performance!

Summary

- The **speed gap** between CPU, memory, and mass storage continues to widen
- Well-written programs exhibit a property called **locality**
- Memory hierarchies based on **caching** close the gap by exploiting locality

Review Questions

- What is an ISA?
- What are the 3 main kinds of commands in an ISA?
- Why do CPUs have both registers and caches?
- Why is it typically impossible for data processing programs to achieve 100% processor utilization?